

Seeeduino XIAO

28June26 © B. Rasnow

Table of Contents

Objective	1
Pinout	1
XIAO Software	2
Configuring Arduino IDE and uploading	4
MATLAB oscscope software (sscope.m)	4
Results	5
Harmonic Distortion	5
Software frequency control	8
Calibrating voltage control (svolts2pwm.m)	12
Discussion	15

Objective

Port the Arduino interfaces to Seeeduino XIAO. This microcontroller is vastly improved over the Arduino. There are lots of variants, the one I tested has the RP2350 CPU (<https://www.seeedstudio.com/Seeed-XIAO-RP2350-p-5944.html>), the same as the Raspberry pi Pico2. Note that there is a hardware bug in the original RPi Pico RP2040 ADC system, that makes it hang up on a few ADU values. This creates havoc with fourier transforms (the mini-plateaus the bug creates add a plethora of high frequency artifacts to the spectra). Some key advantages of the XIAO are

1. Smaller footprint → more solderless breadboard real estate for test circuits.
2. Much faster ADC, perhaps enabling measuring inductors and HF phenomena. Measured 2M voltages per minute below.
3. 3 analog input channels (Nano has 8, Pico only 2).
4. Huge data memory so we can increase NUMSAMPLES to thousands (vs. 400 on Arduino).
5. Better serial communication, to move data faster, more reliably, and no more CH340 driver problems.
6. \$4-5 each.
7. ...

Note, I first ported to the Raspberry pi Pico2, documented elsewhere. The Pico2 is larger than the Nano, and also easy to fry.

Pinout

Like a Pico, the silk screen is underneath, and I used short pins so I can't see them. Here's the pinout.

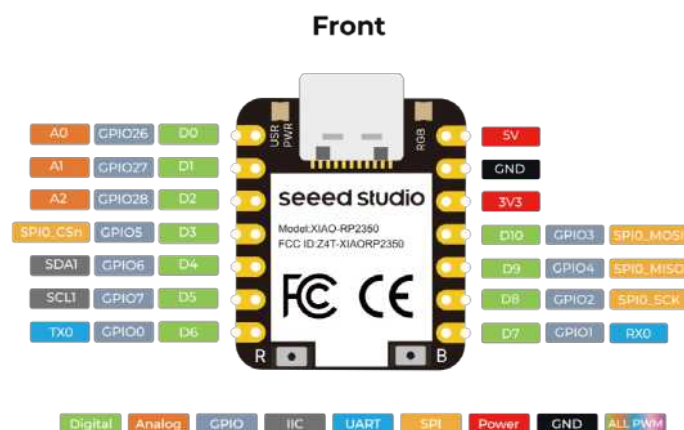


Figure 1. Seeduino XIAO-RP2350 pinout.

There's just one GND pin, so don't forget to connect it to BOTH ground busses on the solderless breadboard (i.e., blue jumper wire on far right in Fig. 2).

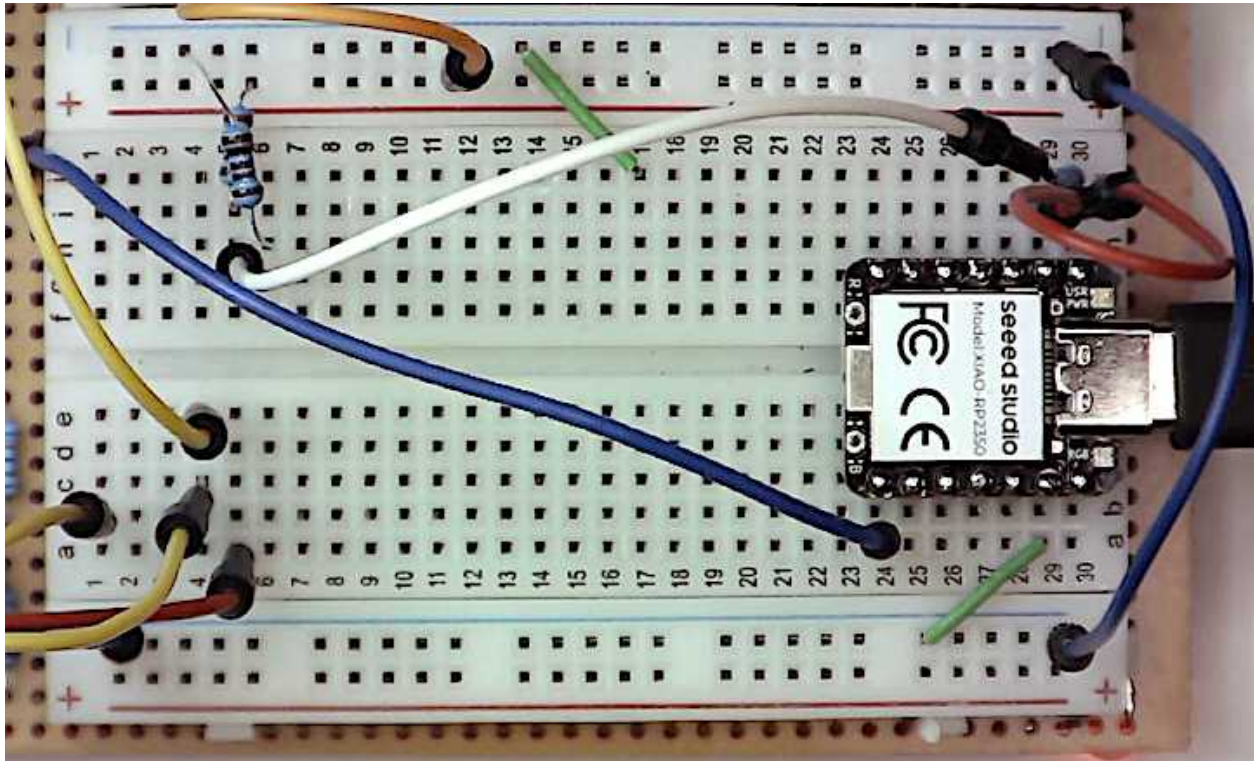


Figure 2. Seeduino on a mini breadboard, voltage divider and 10k resistor between function generator (orange top) -> A0 and A1. Lower left pin -> blue jumper -> PWM filter and current mirror circuit (yellow jumpers).

XIAO Software

Tweaking oscscope – need to renumber pins, and add runtime-programmable delay to slow down data acquisition (discussed below). Note, `pinMode(7, OUTPUT)` is **not** the same as `pinMode(D7, OUTPUT)`, only the later works! (How much debugging time did it take to figure that out?) Thus `const int analogOutputPin` declaration doesn't work, but `#define` does (and beware `//` comments after a `#define` don't work, only `/* */` ... do you know why? The sample size can be huge, unlimited by memory as the Nano is with 400 samples. 5000 samples was chosen to improve statistics without noticeable performance loss. Analog read and write resolution are also compile-time selectable.

```
res = [10 12 14];
for i = 1:3
    fprintf('1ADU = %.1f mV at %d bit resolution\n', 3.3e3 / 2^res(i), res(i));
end
```

```
1ADU = 3.2 mV at 10 bit resolution
1ADU = 0.8 mV at 12 bit resolution
1ADU = 0.2 mV at 14 bit resolution
```

```
clear res i
```

The "noise floor" is around 1mV so the benefit of 14 bit is doubtful. So as you read the code below, appreciate that much time and effort goes into writing, debugging, and validating. AI is making this quicker.

```
/* file: sscope.ino COMMAND SET
   b -- measure interleaved A0, A1 appended with time (microsecs)
   d <microseconds> -- delay <microseconds> between data acquisitions
   p <pwmValue> write <pwmValue> integer 0-1023 to analogOutPin
*/
#define NUMSAMP 5000
#define ANALOG_OUT_PIN D7 /* bottom corner */
uint16_t pwmVal = 0;
uint16_t data[NUMSAMP+1][2], i, microsecDelay=0, nchan=2;
int settleTime = 200; // msec
uint32_t t0, t1;
void measureWaveformsBinary(void);
void setup()
{
    Serial.begin(115200);
    pinMode(A0, INPUT);
    pinMode(A1, INPUT);
    analogReadResolution(12);
    pinMode(ANALOG_OUT_PIN, OUTPUT);
    analogWriteResolution(12);
    analogWrite(ANALOG_OUT_PIN, 0);
    Serial.println("sscope"); // wait for UART to boot
} // setup
void loop()
{
    if (Serial.available())
    {
        switch (Serial.read())
        {
            case 'p': // set pwm value
            {
                pwmVal = Serial.parseInt();
                analogWrite(ANALOG_OUT_PIN, pwmVal);
                delay(settleTime);
                break;
            }
            case 'b': // readAndWriteWaveformsBinary
            {
                measureWaveformsBinary();
                break;
            }
            case 'd': // delay (microseconds) between data acquisitions
            {
                microsecDelay = Serial.parseInt();
                break;
            }
        }
    }
} // switch
```

```

    delay(1);
} // loop
void measureWaveformsBinary(void)
{ // Measure  NUMSAMP values on A0 and secondChan
  // return data and time in binary format
  t0 = micros();    // time it to compute sample rate
  for (i = 0; i < NUMSAMP; i++)
  {
    data[i][0] = analogRead(A0);
    delayMicroseconds(microsecDelay);
    data[i][1] = analogRead(A1);
    delayMicroseconds(microsecDelay);    // even interleave
  }
  t1 = micros();
  data[i][0] = (uint16_t) (t1 - t0); // put dt at end of data
  data[i][1] = (uint16_t) ((t1 - t0) >> 16); // most significant 2 bytes
  Serial.write((uint8_t*)data, (NUMSAMP + 1) * 2 * sizeof(uint16_t));
} // measureWaveformsBinary

```

Configuring Arduino IDE and uploading

https://wiki.seeedstudio.com/xiao_rp2350_arduino/ gives directions. I had no success with a USB-A to C cable plugged into a short USBC-USBA(F) connector (on my Macbook Pro). Tried a Thunderbolt4 cable. Now the 1-button boot shown in the online video got the IDE to acknowledge an "unknown device on /dev/cu.usbmodem2101" (after the OS asked permission to connect a device). After clicking Upload I had to give the OS permission again to connect, and the upload succeeded. Many USBC cables don't support data. A \$20 Thunderbolt (TB4) worked along with a few others, but most USBC cables I had did not work.

MATLAB oscscope software (sscope.m)

Save oscscope4.m → sscope.m (don't forget to change `runBtn.Callback = 'sscope'`). Found what I thought was a bug, unstable triggering, variable frequency and phase. Turns out low frequencies (100's of Hz) vary slow enough that noise was causing false triggering. $1 \text{ ADU} = 3.3\text{V}/2^{12} = 0.8\text{mV} = 800\mu\text{V}$, so it shouldn't be surprising that noise $> 1\text{ADU}$. If the voltage dwells near the threshold for longer than a sample, our simplistic trim algorithm detects multiple periods. Three fixes are to increase the frequency (i.e., frequency jumper in 4th position), reduce the ADC resolution from 12 bits, or add an intersample delay, e.g., `writeline(seed, 'd 20')`. Here's sscope.m:

```

%% sscope.m -- 9jul25 for seeeduino XIAO-RP2350
% from oscscope4, 20 nov 22 putting variables in dat structure to reduce clutter
% seed running sscope.ino, command set = b, p, d
if ~exist('runBtn','var') || ~isvalid(runBtn) % add the button once
    runBtn = uicontrol('style','radiobutton','string','run', ...
        'units','normalized','position',[.03 .94 .1 .04], ...
        'Callback','sscope');
    dat.skew = uicontrol('Style','popupmenu','Units','normalized',...
        'Position',[.1 .01 .18 .05],'String',{'No deskew','Spline','Fourier'});
    dat.trimThresh = 506; % runs once
    dat.runOnce = true;
end
if ~exist('seed','var') % initialize arduino
    disp('initializing seed ...')
    ports = serialportlist;

```

```

    seed = serialport(ports{end},115200);
    seed.Timeout = 2;
    clear ports;
    pause(1);
end

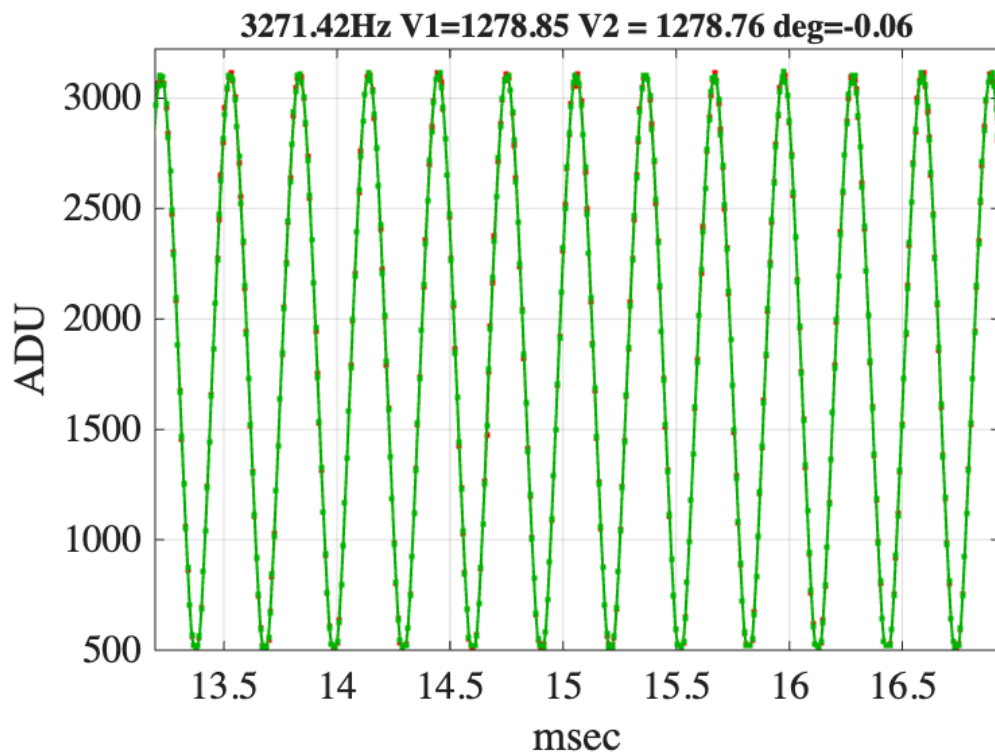
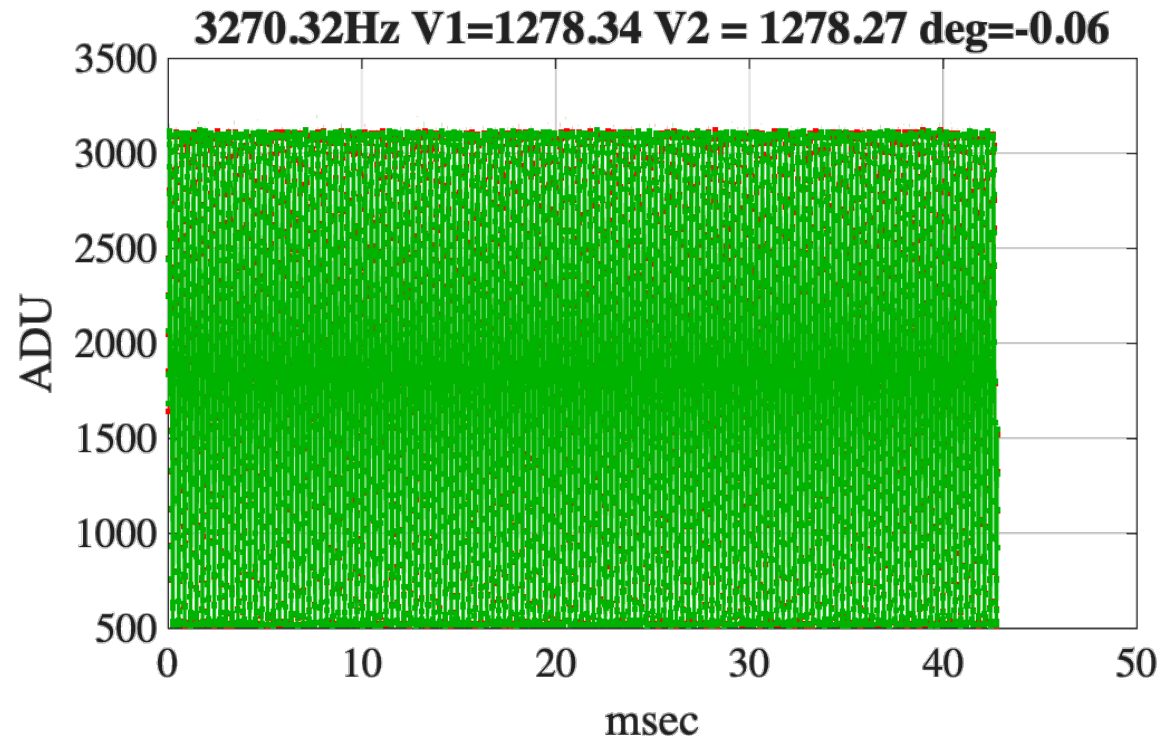
while runBtn.Value || dat.runOnce
    writeline(seed,'b');
    try
        dat.numSamp = 5000; % same as NUMSAMP in the .ino file
        dat.raw = 0; dat.dt = 0; dat.bin = 0;
        dat.bin = read(seed,dat.numSamp*2+2,'uint16');
        dat.dt = bitshift(dat.bin(end),16)+dat.bin(end-1); % microseconds
        dat.raw = reshape(dat.bin(1:end-2),2,dat.numSamp)';
        [dat.data, dat.npds] = trim(dat.raw, dat.trimThresh, -1, 1);
        % 2nd arg of trim ~ mean(dat.raw(:,1))
        dat.t = linspace(0, dat.dt/1000, dat.numSamp)';
        dat.t = dat.t(1:length(dat.data));
        dat.sr = length(dat.raw)/dat.dt*1e6; % sample rate (#/sec)
        dat.freq = dat.sr/(length(dat.data)/dat.npds); % Hz, 1/sec
        dat.data = arddes skew(dat.data, dat.skew.String{dat.skew.Value});
        dat.fd = fft(dat.data); % data was trimmed to npds
        dat.ampl = dat.fd(dat.npds+1,:) / length(dat.data);
        if dat.npds ~= 0, dat.ampl = dat.ampl * 2; end % normalization for AC
        dat.theta = rad2deg(angle(dat.ampl));
    catch
        flush(seed);
        fprintf('*');
        break;
    end
    npts = 500; % number of samples to plot
    plot(dat.t(1:ns), dat.data(1:ns,:), '.-');
    % aspectrum(dat.data, dat.sr);
    xlabel('msec'); ylabel('ADU'); grid on;
    title(sprintf('%.2fHz V1=%.2f V2 = %.2f deg=%.2f', ...
        dat.freq, abs(dat.ampl), diff(dat.theta)), 'FontSize', 12);
    if runBtn.Value, pause(.1); end % otherwise it is just too fast ...
    displayfps;
    dat.runOnce = false;
end

```

Results

Harmonic Distortion

```
dat.runOnce = true; sscope;
```

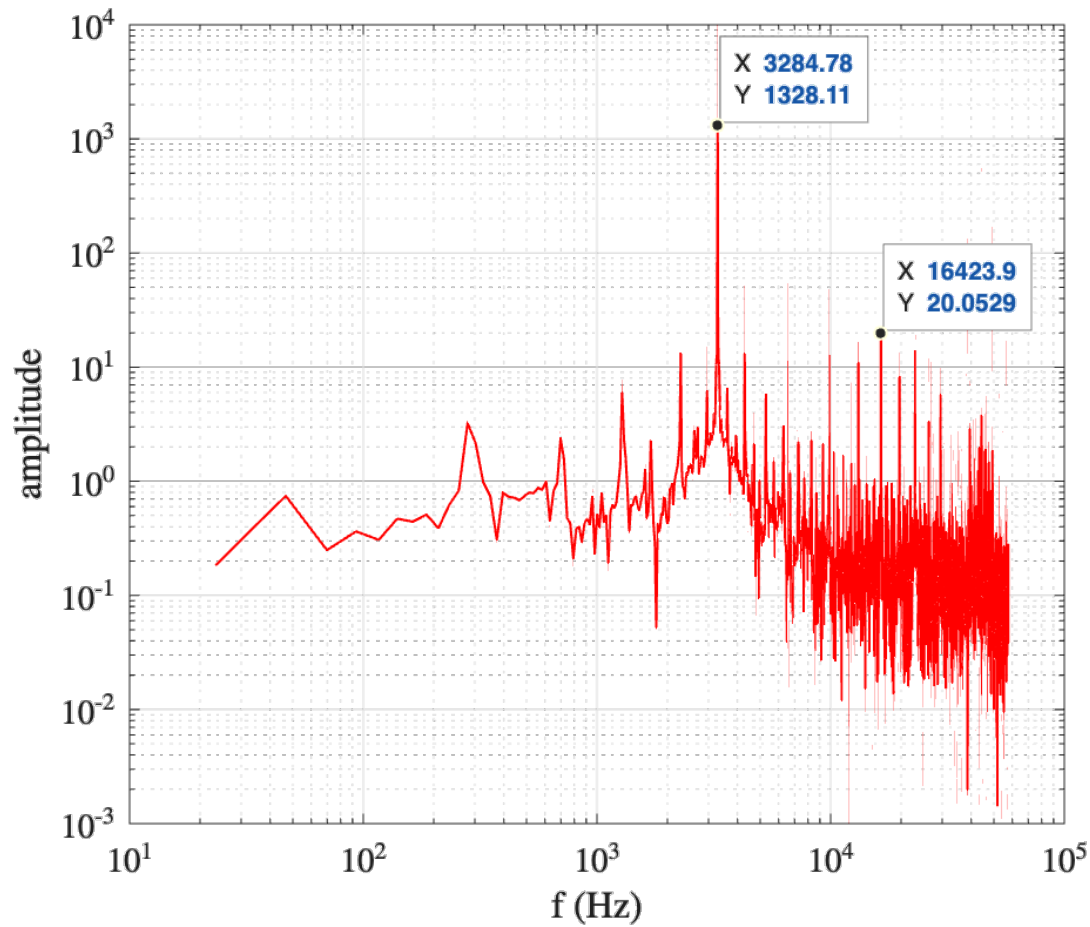


Figures 3 and 4: sscope display and zoomed in.

```
dat.sr
```

```
ans = 1.1502e+05
```

spectrum:



I verified most of the peaks to the right are harmonics – except the symmetric pair adjacent to the main peak. Harmonic amplitudes are, and the adjacent peaks are symmetrical at frequencies:

```
harmonicAmpls = abs(dat.fd(dat.npds*(1:6)+1,1))'/length(dat.data)*2
```

```
harmonicAmpls = 1x6  
1.0e+03  
1.3281    0.0111    0.0126    0.0109    0.0201    0.0082
```

```
df_of_nearest_peaks = [2283.04 4286.52] - 3284.78 % with amplitudes 13.18 and 13.01ADU
```

```
df_of_nearest_peaks = 1x2  
1.0e+03  
-1.0017    1.0017
```

```
THD = sqrt(sum(harmonicAmpls(2:end).^2)) / harmonicAmpls(1)
```

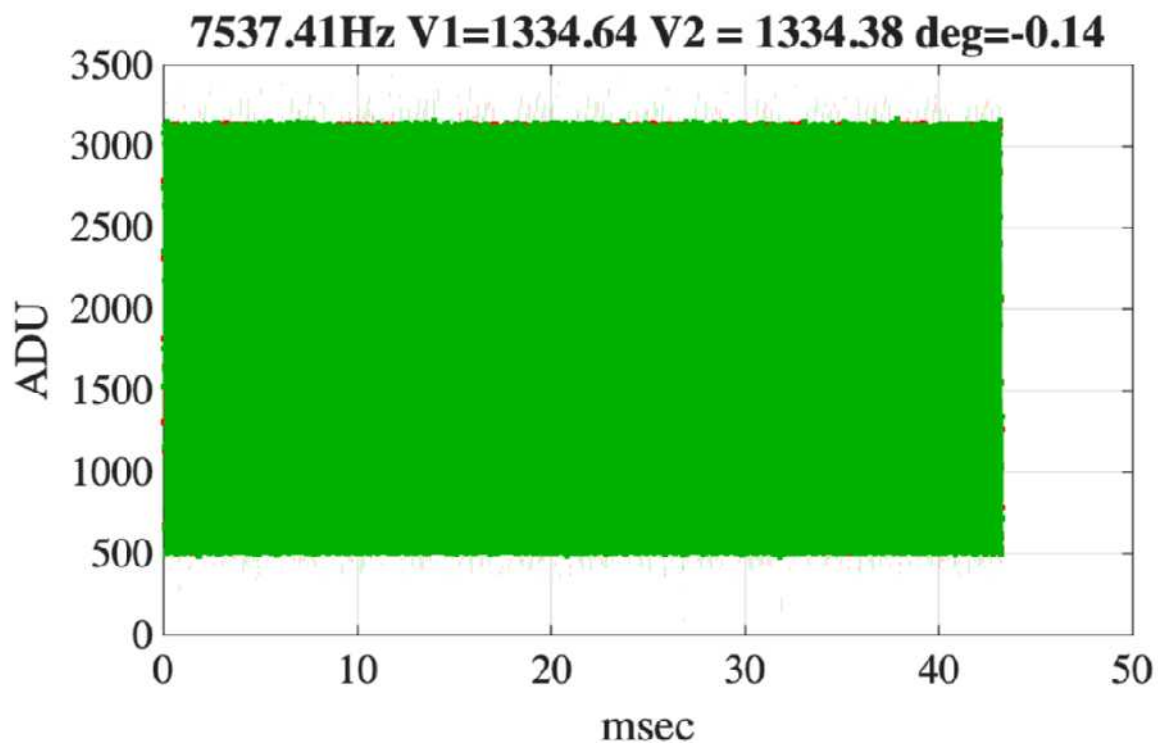
THD = 0.0222

which is as good as I've ever seen this XR2206 design perform – probably because sine shaping is better at this high a frequency.

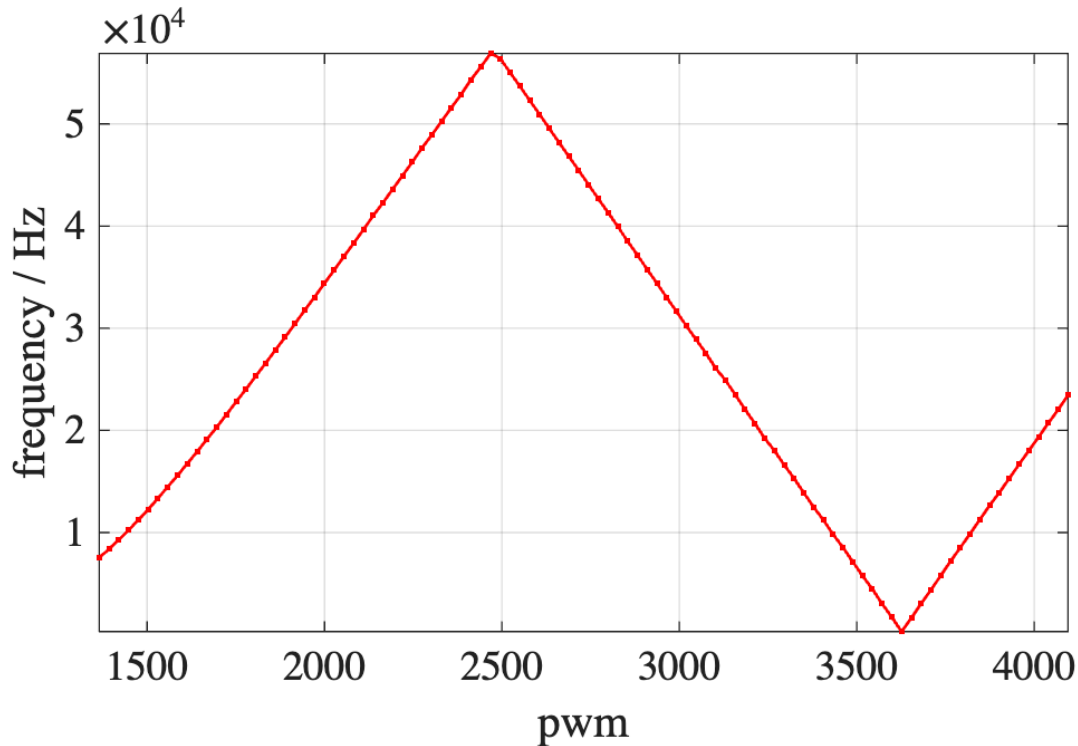
Software frequency control

The range of PWM values is now 0-4095 corresponding to ~0-3.3V, so the current mirror pot has to be reduced. Connected the LM317 to the current mirror, `writeline(seed,'p 4095')`, and tweaked the pot until the voltage ~ 1.3V. Next connected the mirror to the function generator shunt jumper.

```
flush(seed) % force buffer to empty
pwmVal = linspace(4095,1.1/3.3*4095,100)';
writeline(seed,'p 4095'); pause(2); % ideally pause 1 minute!
fmeas = zeros(size(pwmVal));
tic;
for i=1:length(pwmVal)
    writeline(seed,['p' num2str(pwmVal(i))]);
    pause(.1); % time for the frequency to stabilize
    dat.runOnce = true;
    sscope; % --> new data in dat
    fmeas(i) = dat.freq;
end
```



```
writeline(seed,'p 4095'); % expect to repeat ...
plot(pwmVal, fmeas, '-'); axis tight; grid;
xlabel('pwm'); ylabel('frequency / Hz');
```



```
toc
```

Elapsed time is 28.980919 seconds.

Double aliasing ... not too different than with the Arduino. I see 4 solutions: 1) restrict the PWM values below ~2500, 2) tweak the function generator's capacitor just like we did for Arduino, 3) unalias – unreflect, i.e., for PWM on the negative slope, `freq -> dat.sr - freq`, and on the right side, `freq -> dat.sr + freq`. (BTW, the phase shift jumps from ~0 to $\pm 180^\circ$ above the first reflection.) 4) Further reduce the current mirror pot value, increasing the mirrored current. The XR2206 I recall (I think) can sink 3mA, so we could get 3x the frequency range. The actual frequency range was

```
fprintf('%.1kHz < freq < %.1kHz\n', fmeas(end)/1e3, (dat.sr+fmeas(1))/1e3)
```

```
7.3kHz < freq < 131.4kHz
```

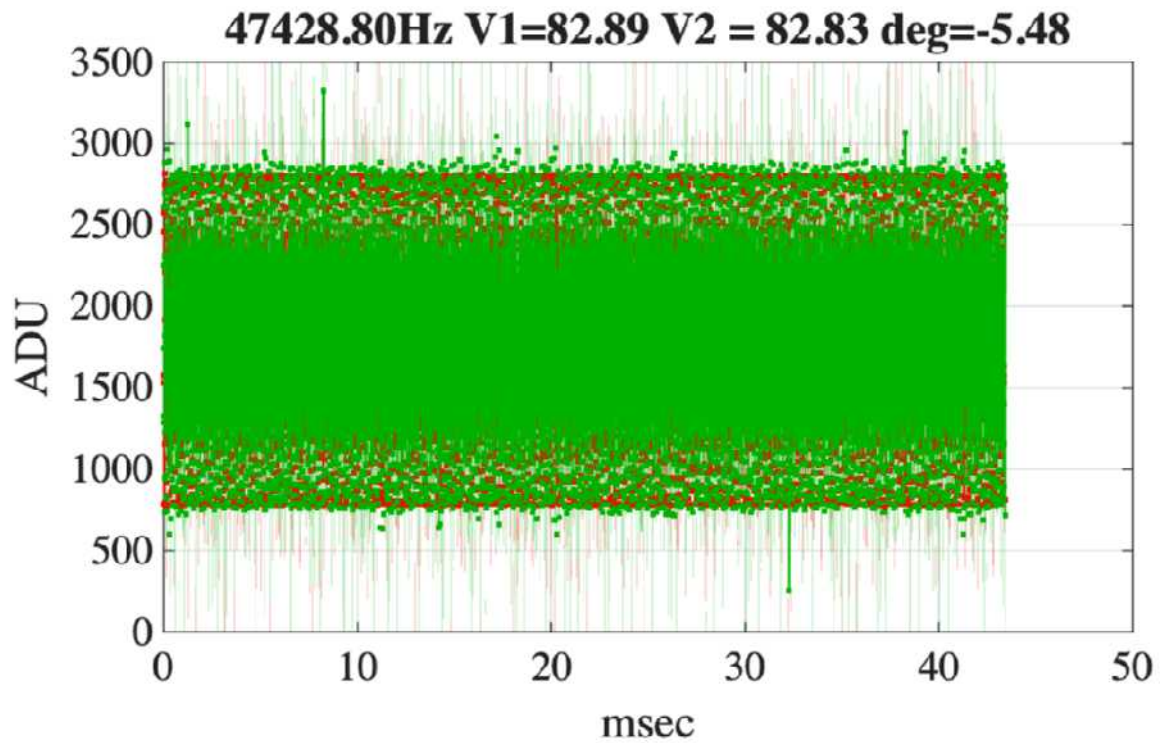
which have a ratio of 18:1, same as Arduino. Cranking down the current mirror pot randomly,

```
clf;
fmeas = zeros(size(pwmVal));
dphase = fmeas;
tic;
for i=1:length(pwmVal)
    writeline(seed,['p' num2str(pwmVal(i))]);
```

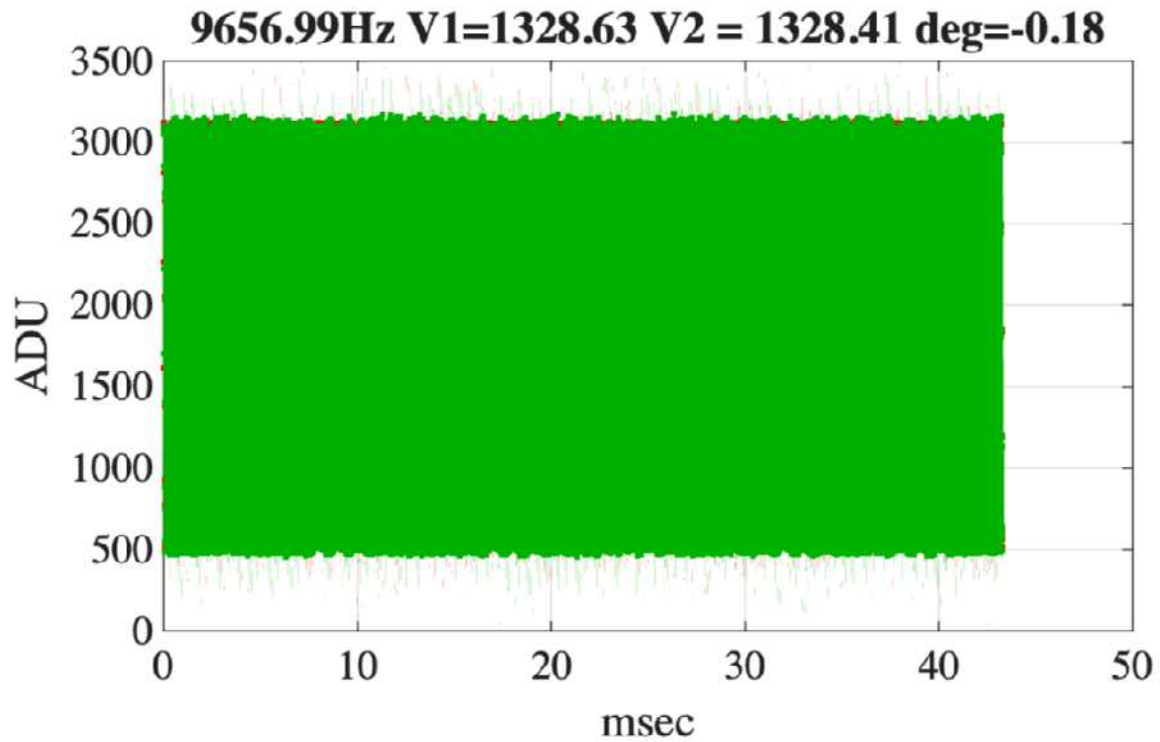
```

pause(.1); % time for the frequency to stabilize
dat.runOnce = true;
ssscope; % --> new data in dat
fmeas(i) = dat.freq;
dphase(i) = diff(dat.theta);
end

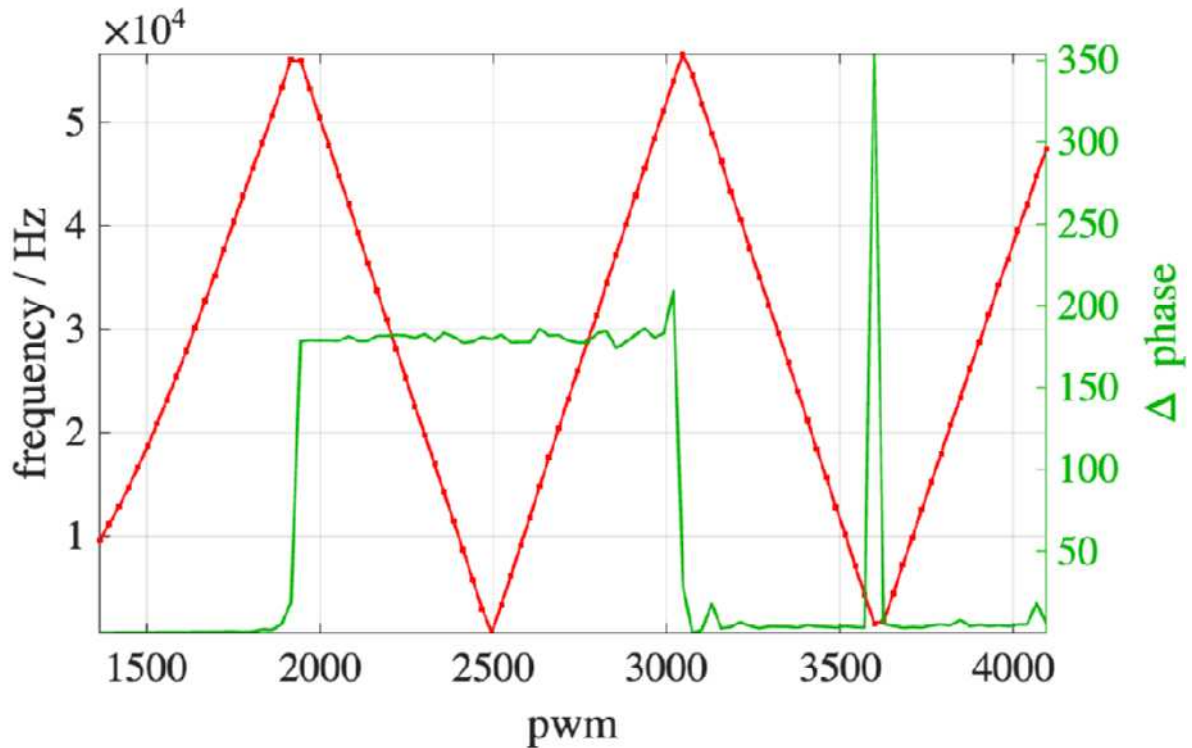
```



Warning: An error occurred while drawing the scene: SceneModel error in command compositeCommand: Typ
 at <https://127.0.0.1:31517/toolbox/matlab/uitools/figurelibjs/release/bundle.mwBundle.gbtfigure-l>



```
writeline(seed,'p 4095'); % expect to repeat ...
plot(pwmVal, fmeas, '-'); axis tight; grid;
xlabel('pwm'); ylabel('frequency / Hz');
yyaxis right;
plot(pwmVal,abs(dphase));
ylabel('\Delta phase')
```



Phase behaves this way because it's the same signal on 2 channels. 4 reflections means the frequency range is

```
fprintf('%.1kHz < freq < %.1kHz\n', fmeas(end)/1e3, (2*dat.sr+fmeas(1))/1e3)
```

```
9.7kHz < freq < 277.4kHz
```

That's a frequency ratio of 28:1, not bad! A cool thing is that we can measure practical inductors and resonances in this frequency range – that just wasn't feasible with Arduino. The above graph (and prior one) are distilled from 1 million voltage measurements, taken in 29 seconds (that includes 10 seconds of pauses).

Calibrating voltage control (svolts2pwm.m)

Adding 330 ohms in series with voltage divider R1 keeps the ADC from clipping – because the ADC vref = 3.3V, so we need to divide LM317's output by a little more than 3.

```
clf;
flush(seed); % just in case left over data in fifo buffer
vref = 3.297; % volts on XIA0 3V3 pin
% don't forget to connect XIA0 ground pin to LM317 ground!
R1 = 1981 + 326; % ohms dividing Vin
R2 = 1003; % ohms dividing Vin
% not expecting change below 1.2V/3.3V*4095adu = 1500adu
pwm = round(linspace(1300,4095,50));
writeline(seed, 'p 0'); pause(1); % extra settling time
v317 = zeros(size(pwm)); % allocate memory for voltage readback
noise = v317;
```

```

tic; % start a timer
for i=1:length(pwm)
    writeline(seed,sprintf('p %d',pwm(i)));
    dat.runOnce = true;
    sscope; % --> new data
    title(num2str(pwm(i)));
    v317(i) = mean(dat.data(:,1)) / 4095 * vref * (R1+R2)/R2;
    noise(i) = std(dat.data(:,1)) / 4095 * vref * (R1+R2)/R2;
end
writeline(seed,'p 0');
toc

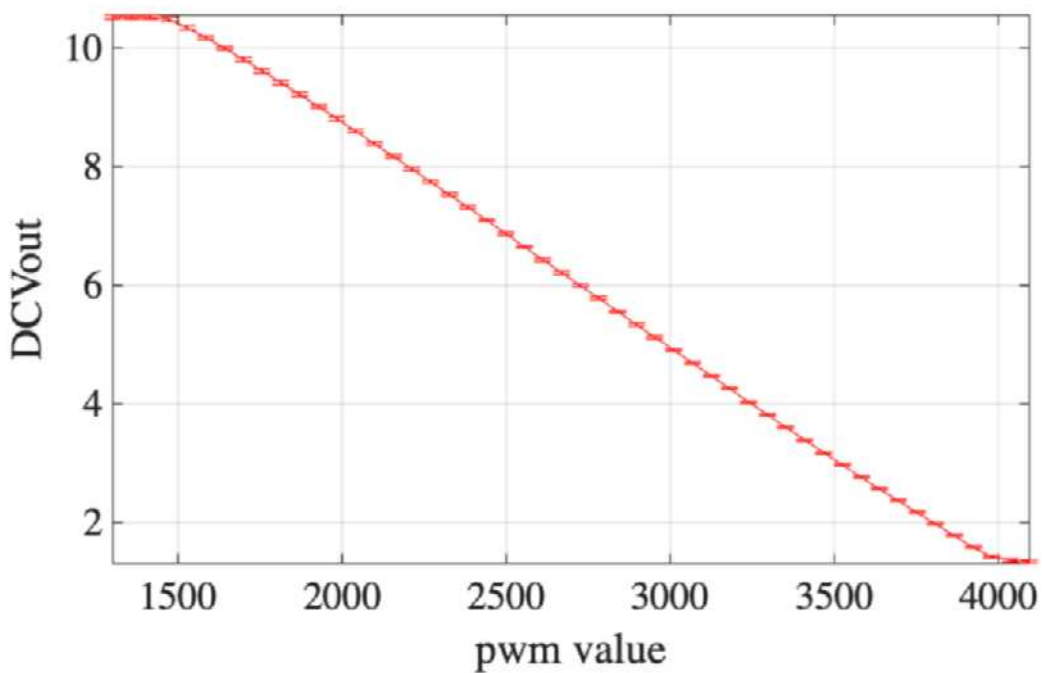
```

Elapsed time is 14.248755 seconds.

```

errorbar(pwm,v317,noise);
xlabel('pwm value'); ylabel('DCVout'); grid; axis tight;

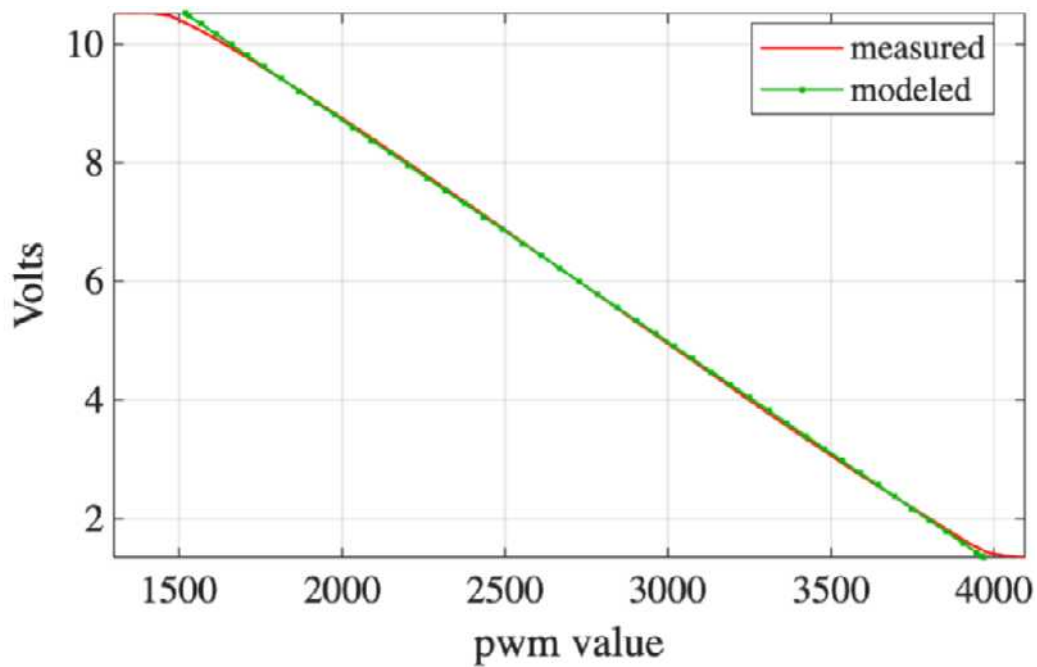
```



```

indx = v317 < .95 * max(v317) & v317 > 1.05 * min(v317); % boolean
pv = polyfit(v317(indx), pwm(indx), 1); % flipped x and y
pwmModel = polyval(pv,v317); % model the polynomial
plot(pwm, v317, pwmModel, v317, '-.')

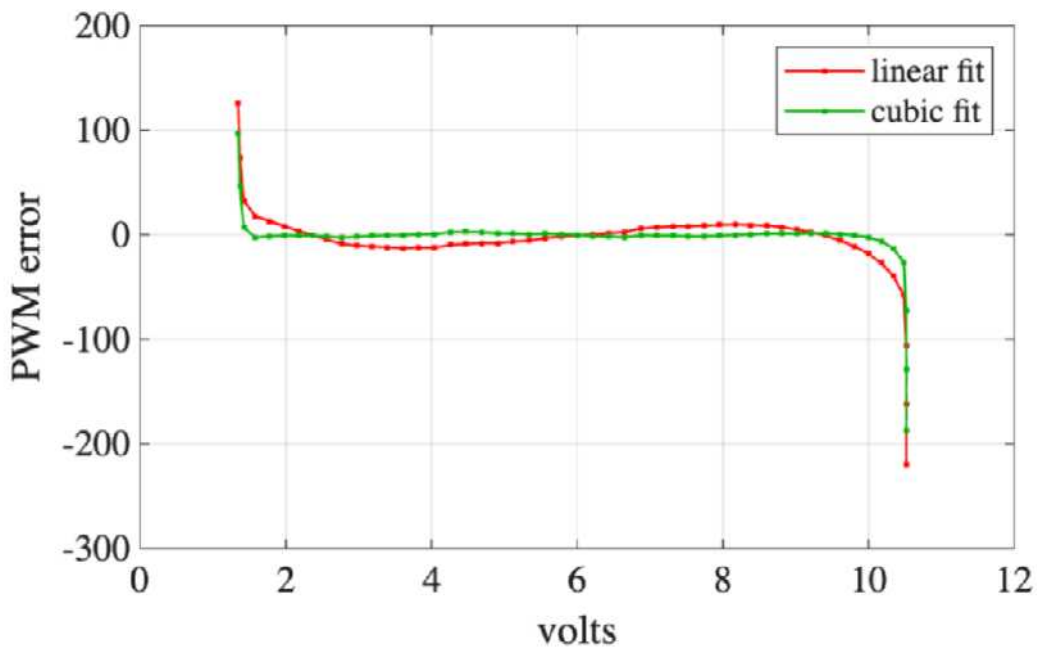
```



```

xlabel('pwm value'); ylabel('Volts');
legend('measured','modeled','location','best')
axis tight; grid
err = pwm - pwmModel;
pv3 = polyfit(v317(indx), pwm(indx), 3); % flipped x and y from Fig. 9
pwmModel3 = polyval(pv3,v317);
err3 = pwm - pwmModel3;
plot(v317, [err err3], '-.-'); grid;
xlabel('volts'); ylabel('PWM error');
legend('linear fit','cubic fit')

```



```
sprintf('%.3e ',pv3)
```

```
ans = '-7.342e-01 1.301e+01 -3.352e+02 4.427e+03 '
```

Not a well balanced polynomial – think about how you might balance it better? \pause

But good enough for svolts2pwm.m:

```
function [pwmVal] = svolts2pwm(volts, seed)
% function [pwmVal] = svolts2pwm(volts, seed)
% return pwm value corresponding to volts for LM317 Seeeduino XIAO ...
% if a serial port is passes as 2nd arg then sends p command
% XIAO configured for 12 bit DAC and ADC, its vref is close to 3.3V

pv = [-7.342e-01 1.301e+01 -3.352e+02 4.427e+03]; % 30dec25 BR
pwmVal = round(polyval(pv, volts));
pwmVal(pwmVal < 0) = 0;
pwmVal(pwmVal > 4095) = 4095;
if nargin == 2
    writeline(seed,['p' num2str(pwmVal)]);
end
end
```

Discussion

It's hard to think of any reason *not* to switch from Arduino Nano to XIAO RP2350 (or perhaps another flavor). The XIAO is much more robust to transient overvoltage than a Raspberry Pi Pico, which was very unforgiving and easy to destroy. The software is very slightly more complex, as there are options to specify regarding bits per sample. It's also necessary to slow the data acquisition with another delay. Larger buffer and faster sampling yields better spectra with less aliasing. Larger data buffers could enable more sensitive measurements including HF or inductive resonant circuits, RF, etc.